Recursion

Bit by Bit Coding - Term 2 | Week 4

Key Concepts

- A recursive function calls itself to solve a smaller version of the problem.
- Base case: the stopping condition (prevents infinite recursion).
- Recursive case: the function calls itself with a smaller input.
- Each call is placed on the call stack until the base case is reached.
- Recursion can replace loops for naturally self-similar problems.

Syntax Reference

Structure of a Recursive Function

```
def recurse(n):
    if n <= 0:                  # BASE CASE
        return 0
    return recurse(n - 1) + n   # RECURSIVE CASE
```

Factorial

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)
print(factorial(5))   # 120  (5*4*3*2*1)
```

Fibonacci & Exponentiation

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)   # fib(6)=8

def power(base, exp):
    if exp == 0:
        return 1
    return base * power(base, exp - 1)  # power(2,3)=8
```

Flattening a Nested List

```
def flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):
            result += flatten(item)
        else:
            result.append(item)
    return result
```

Call Stack Visualization

- factorial(3) calls factorial(2) calls factorial(1).
- Each call waits until the one below it returns.
- Results bubble back up: 1 -> 1*2=2 -> 2*3=6.

Common Patterns

- Ask: "What is the simplest version?" - that's the base case.
- Assume the function works for smaller inputs, then build up.
- Watch for deep recursion - Python has a recursion limit (~1000).

Tips & Common Mistakes

- No base case = infinite recursion -> RecursionError.
- The base case must actually be reached (input must shrink each call).
- Recursion can be slower than loops; use it when the problem is naturally recursive.
- Any recursion can be written as a loop - pick the clearer one.